

Week 3-4 — Classical ML From Scratch + VAD Project

Same format as before: spec-level steps, "done when" checks, resources per day. Write every line yourself — libraries (sklearn, librosa) show up here only as a way to *check* your from-scratch work afterward, never to do the work for you.

WEEK 3 — Classical ML From Scratch

Day 15 — Linear regression

Steps:

1. Generate a toy dataset: $y = 3x + 2 + \text{noise}$ using numpy, so you know the ground-truth coefficients in advance.
2. Implement the closed-form **normal equation**: augment X with a column of ones (for the bias/intercept), then $w = (X^T X)^{-1} X^T y$.
3. Implement **gradient descent**: loss $= (1/n) ||Xw - y||^2$, gradient $= (2/n) X^T (Xw - y)$ — this is the exact formula from your Day 2 math reference doc ($(\partial (||Ax-b||^2)/\partial x)$), go use it directly rather than re-deriving from scratch.
4. Confirm both methods converge to (approximately) the same w .
5. **Stress-test the normal equation**: construct a dataset with two nearly-identical (multicollinear) features, so $X^T X$ is close to singular. Watch the normal equation produce unstable/wildly-scaled coefficients while gradient descent still degrades more gracefully. This is Day 1's invertibility material ($\det(A) \approx 0 \Rightarrow \text{near-singular} \Rightarrow \text{unstable inverse}$) showing up as a real, visible failure mode, not just theory.

Resource: StatQuest with Josh Starmer — search the channel for "Linear Regression."

Done when: both methods agree on the well-conditioned dataset, and you've watched (and can explain) the normal equation degrade on the multicollinear one.

Day 16 — Logistic regression

Steps:

1. Derive the gradient by hand first: for binary cross-entropy loss with $\hat{y} = \sigma(Xw)$, the gradient works out to the clean form $(\partial L / \partial w = (1/n) X^T (\hat{y} - y))$ — structurally the same "predicted minus actual" cancellation you already saw with softmax + cross-entropy in Week 2. Notice that pattern, it's not a coincidence.
2. Implement a numerically stable sigmoid — very large negative inputs can underflow/overflow the plain $1/(1+\exp(-z))$ formula; handle it the way you handled log-sum-exp in Week 2 (case-split on the sign of z), or use `scipy.special.expit` only as a reference to check against, not to import into your own code).
3. Implement the gradient descent training loop.
4. Train on a toy 2D dataset, plot the learned decision boundary.

5. Verify your hand-derived gradient formula against your Week 1 numerical gradient checker.

Resource: StatQuest — search "Logistic Regression."

Done when: the numerical gradient check confirms your hand-derived formula, and the decision boundary plot looks sane on a linearly-separable toy dataset.

Day 17 — k-NN

Steps:

1. Implement a distance function (Euclidean to start).
2. Implement brute-force k-NN: for a query point, compute distance to every training point, take the k closest, return the majority class.
3. Test on a small toy dataset.
4. **Run the curse-of-dimensionality experiment yourself**, don't just read about it: generate N random points uniformly in a d -dimensional cube, compute the ratio of (distance to nearest neighbor) / (distance to farthest neighbor), and repeat for increasing d (e.g. $d = 2, 10, 50, 200$). Plot how that ratio changes. You should see it creep toward 1 as d grows — meaning "nearest" and "farthest" stop being meaningfully different, which is why k-NN degrades in high dimensions. Seeing the number move is a very different kind of understanding than reading the sentence "k-NN suffers from the curse of dimensionality."

Resource: StatQuest — search "K Nearest Neighbors."

Done when: your classifier works, and you have an actual plot showing the distance-ratio collapse from the experiment in step 4.

Day 18 — k-means

Steps:

1. Implement the standard loop: random init of k centroids $\rightarrow$ assign each point to its nearest centroid $\rightarrow$ recompute each centroid as the mean of its assigned points $\rightarrow$ repeat until centroids stop moving (or a max iteration cap).
2. Plot cluster assignments at a few points during convergence (before, middle, after) to see it visually settle.
3. Implement **k-means++ initialization**: pick the first centroid uniformly at random, then pick each subsequent centroid with probability proportional to its squared distance from the nearest already-chosen centroid.
4. **Demonstrate the initialization-sensitivity problem directly**: construct a dataset with clusters of very different sizes (e.g. one large, two small and close together), run plain-random-init k-means many times, and show it sometimes lands in a visibly bad local optimum (e.g. splitting the large cluster instead of separating the two small ones). Then show k-means++ does this less often over the same number of runs.

Resource: StatQuest — search "K-means clustering."

Done when: you have a concrete example where random init visibly fails and k-means++ visibly does better, not just the algorithm running once successfully.

Day 19 — Decision tree

Steps:

1. Implement Gini impurity (or entropy — pick one, Gini is simpler to start).
2. Implement best-split search: for each feature, for a set of candidate thresholds, compute the weighted impurity of the resulting left/right split, and track the best `(feature, threshold)` pair found.
3. Implement recursive tree construction: a `Node` holds either a `(feature, threshold, left_child, right_child)` for internal nodes or a predicted class for leaves. Recurse until a stopping condition (max depth, minimum samples per leaf, or a pure node).
4. Implement prediction: walk from the root, following left/right based on each node's learned threshold.
5. **Demonstrate overfitting directly:** train one tree with no depth limit and one with a shallow depth cap on the same data. The unbounded tree should hit ~100% training accuracy (it can memorize), while its held-out accuracy is worse than the shallow tree's. This is the concrete, visible version of the bias-variance tradeoff you'll be asked about in interviews.

Resource: StatQuest — search "Decision Trees" (multi-part series, work through the classification parts).

Done when: the tree correctly classifies its own training data when unconstrained, and you've measured (not just stated) the overfitting gap between unbounded and depth-capped versions on held-out data.

Day 20 — PCA

Steps:

1. Center the data (subtract the mean of each feature).
2. Compute the covariance matrix.
3. Eigendecompose it — use `np.linalg.eigh` (not the general `eig`) since the covariance matrix is symmetric, which is exactly the Day 1 spectral theorem guaranteeing real eigenvalues and orthogonal eigenvectors, and `eigh` is the numerically appropriate routine for that case.
4. Sort eigenvectors by eigenvalue, descending; the top `k` are your principal components.
5. Project the centered data onto those top-`k` components.
6. **Cross-check via SVD:** compute PCA a second way, via `X = UΣVT` on the centered data — the right singular vectors `V` should match your eigenvectors (up to sign flips, which are mathematically inconsequential). Confirming both routes agree is a real correctness check, not busywork.

7. Compute the explained variance ratio per component ($\frac{\text{eigenvalue}_i}{\sum(\text{all eigenvalues})}$).
8. **Apply it to your own MNIST data from Week 2** — reduce a subset of digit images to 2D and scatter-plot them colored by digit label. You should see meaningful (if imperfect) separation between some digit classes.

Resource: StatQuest — search "PCA main ideas" and "PCA step-by-step" (both parts are worth watching).

Done when: eigendecomposition and SVD give matching components, and your 2D MNIST projection visibly shows class structure, not a random cloud.

Day 21 — Review and consolidate

Write a reference doc: for each of the six algorithms above, one paragraph covering time complexity, key assumptions, when you'd choose it over the alternatives, and its main failure mode. This is your direct answer key for "when would you use X over Y" interview questions.

WEEK 4 — VAD Project

Day 22 — Research phase

Steps:

1. Watch 3Blue1Brown's Fourier Transform video (search the channel for "But what is the Fourier Transform?") — conceptual intuition before you touch any spectral-feature code.
2. Read the MFCC tutorial fully once, don't implement yet:
<http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfccs/>
3. Understand energy-based VAD conceptually: frame audio into short windows, measure short-term energy per frame, threshold.
4. Skim `py-webrtcvad`'s README to understand what the WebRTC approach does at a high level (small Gaussian mixture models classifying speech vs. non-speech from spectral features): <https://github.com/wiseman/py-webrtcvad>
5. Write a short design doc predicting which of your three planned approaches (energy / GMM / NN) will perform best, and why — before building anything. This is a calibration exercise: you'll check this prediction against your actual Day 27 benchmark results.

Done when: the design doc with your prediction is written, before any VAD code exists.

Day 23 — Baseline: energy-based VAD

Steps:

1. Get sample audio (record your own voice with some silence gaps, or pull a couple of short public speech clips).

2. Frame the audio into short overlapping windows — standard speech-processing framing is ~25ms windows with ~10ms hop (i.e. windows overlap).
3. Compute short-term energy per frame (sum of squared sample values in that window).
4. Threshold: classify a frame as speech if its energy exceeds some threshold. Don't hardcode a fixed number — estimate a noise floor from the first ~10 frames (assumed silence) and set the threshold relative to that.
5. Smooth the decisions: raw per-frame decisions will flicker rapidly between speech/silence at boundaries. Add a simple "hangover" rule — require a few consecutive frames of the new state before actually switching.
6. Plot the waveform with your VAD decision overlaid as shading.

Done when: the overlay plot looks visually sensible on your sample audio — silence gaps correctly marked, speech correctly marked, without excessive flickering.

Day 24 — Feature extraction (MFCC)

Steps:

1. Pre-emphasis filter — a simple high-pass filter that boosts higher frequencies (speech has less energy there naturally, this compensates).
2. Frame + window each segment (Hamming window, to reduce spectral leakage at frame edges).
3. FFT each frame, take the power spectrum.
4. Build a Mel filterbank: convert Hz to Mel scale, construct overlapping triangular filters spaced evenly in Mel space, apply to the power spectrum.
5. Take the log of the filterbank energies.
6. Apply a DCT to decorrelate them into final MFCC coefficients (typically keep the first 12-13).
7. **Validate against a library, after you've built your own:** extract MFCCs from the same audio with `librosa.feature.mfcc`, compare shapes and general value ranges against your implementation. They won't match exactly (implementation details like filterbank spacing vary) but should be in the same ballpark — this is a "did I get the pipeline roughly right" check, not a requirement to match bit-for-bit.

Resource: <http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfccs/> (same one from Day 22 — this time actually implement each section as you go through it, rather than just reading).

Done when: your from-scratch MFCC output shape and general magnitude are consistent with `librosa`'s output on the same audio file.

Day 25 — GMM-based VAD

Steps:

1. Extract MFCC features across your sample audio, and get (even rough) speech/silence labels per frame — you can bootstrap these from your Day 23 energy-based baseline's most confident decisions.
2. Fit a Gaussian model per class from the labeled features: compute the mean vector and covariance matrix of the speech-labeled frames' MFCCs, and separately for silence-labeled frames. (This is a direct application of the Gaussian MLE formulas from your Day 3 math reference doc — go use them.)
3. Classify new frames by comparing likelihood under the speech-Gaussian vs. the silence-Gaussian, picking whichever is higher.
4. **Stretch goal, if time allows:** replace the single-Gaussian-per-class model with a true Gaussian Mixture Model (multiple Gaussians per class, fit via EM) — closer to what WebRTC's actual VAD does, and a good chance to implement the EM algorithm itself from scratch.
5. Compare its decisions against your Day 23 energy baseline on a few tricky segments (quiet speech, background noise) where you'd expect energy alone to fail.

Done when: the GMM-based classifier runs end to end and produces at least one segment where it visibly outperforms the plain energy baseline.

Day 26 — NN-based VAD

Steps:

1. Reuse your own vectorized `Tensor`/MLP code from Week 1-2 — this is a direct, deliberate callback, not new infrastructure. Binary classification (speech vs. silence) on MFCC feature vectors as input.
2. Train on your labeled frames.
3. Evaluate on held-out frames.

Done when: the network trains (loss decreasing) and produces sane accuracy on held-out data — this one should be the fastest of the three to build, precisely because you already built and understand the underlying engine.

Day 27 — Benchmark all three

Steps:

1. Build one evaluation harness that runs all three approaches (energy, GMM, NN) against the same held-out labeled test set.
2. Measure: accuracy, false-positive rate, false-negative rate, and per-frame processing latency, for each.
3. Compile into a single comparison table.
4. Go back to your Day 22 prediction — were you right about which approach would win? Write a sentence on why or why not; that gap between prediction and result is worth understanding, not just noting.

Done when: you have a real table with real numbers for all three approaches, not just "NN worked best" as a vibe.

Day 28 — Polish and write-up

Write the README: architecture choice and why, tradeoffs weighed, what you'd improve with more time. Mirrors the exact judging criteria from the real interview data you found earlier (accuracy, code quality, improvements you'd have made) — write it like you're being scored against that rubric, because functionally you are.

What's next

Week 5 starts transformers from scratch (attention mechanics), building on your Week 1-2 engine again. Don't start until Day 28's README is actually written.